

## LAB\_8

1. Write the programme to open a text file named input 2, and copy its contents to an output text file output 2.

**CODE:- package LAB\_8;**

**import java.io.File;**

**import java.io.FileReader;**

**import java.io.FileWriter;**

**import java.io.IOException;**

**public class InputOutput {**

**public static void main(String[] args) throws IOException {**

**// TODO Auto-generated method stub**

**//giving the path of the file in which we written the information**

**File inputFile= new File("C:\\Users\\ANKIT  
SINGH\\OneDrive\\Desktop\\java\\inputfile.txt");**

**//giving the path of the file where we paste the information**

**File outputFile=new File("C:\\Users\\ANKIT  
SINGH\\OneDrive\\Desktop\\java\\outputfile.txt");**

**//for reading the data from the file**

**FileReader a=new FileReader(inputFile);**

```
//for writing the data
```

```
FileWriter b=new FileWriter(outputFile);
```

```
//for searching the information
```

```
int c;
```

```
while((c=a.read())!=-1)
```

```
    b.write(c);
```

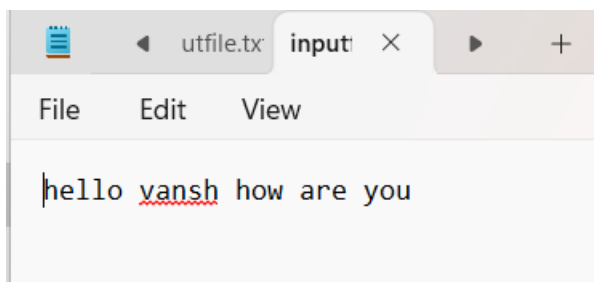
```
a.close();
```

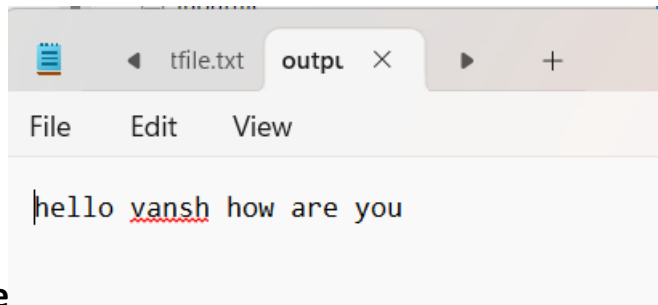
```
b.close();
```

```
}
```

```
}
```

**OUTPUT:-inputfile**





**outputfile**

---

**2.** Write the programme to show multithreading for the string “multi threads”. Show the resulting output.

CODE:- **package** LAB\_8;

**public class** MultiThread {

**public static void** main(String[] args) {

String str = "multi threads";

// Create and start threads

Thread thread1 = **new** Thread(**new** PrintSubstring(str, 0, 5));

Thread thread2 = **new** Thread(**new** PrintSubstring(str, 6, 12));

thread1.start();

thread2.start();

**try** {

// Wait for threads to complete

```

        thread1.join();
        thread2.join();
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}

```

// Runnable implementation to print a substring

```

class PrintSubstring implements Runnable {
    private String str;
    private int start;
    private int end;

    public PrintSubstring(String str, int start, int end) {
        this.str = str;
        this.start = start;
        this.end = end;
    }

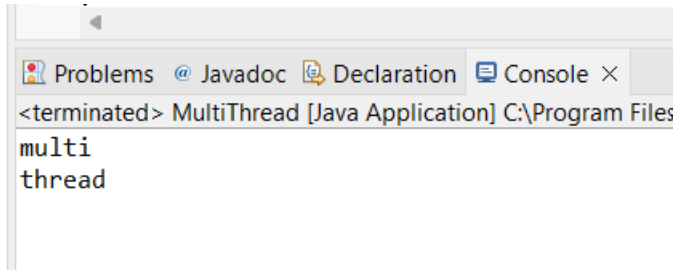
    @Override
    public void run() {
        // Print the substring from the start index to the end index
        System.out.println(str.substring(start, end));
    }
}

```

```
}
```

```
}
```

OUTPUT:-



---

3. Implement a Java program that creates a thread using the Runnable interface. The thread should print numbers from 1 to 10 with a delay of 1 second between each number.

CODE:- **package** LAB\_8;

**public class** RunnableInterface **implements** Runnable{

    @Override

**public void** run() {

        // Loop to print numbers from 1 to 10

**for** (**int** i = 1; i <= 10; i++) {

            System.**out**.println(i);

**try** {

            // Delay of 1 second (1000 milliseconds)

            Thread.sleep(1000);

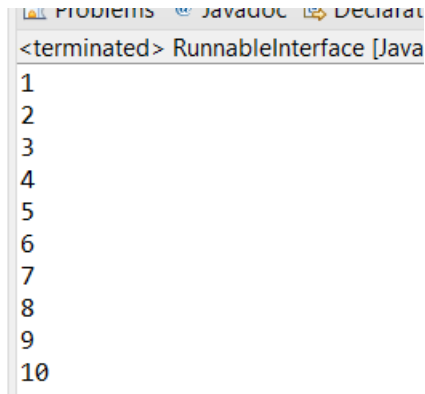
        } **catch** (InterruptedException e) {

            e.printStackTrace();

```
    }  
    }  
}
```

```
public static void main(String[] args) {  
    // Create an instance of printing number  
    Runnable numberPrinter = new RunnableInterface();  
  
    // Create a thread with the printing number instance  
    Thread thread = new Thread(numberPrinter);  
  
    // Start the thread  
    thread.start();  
  
    // waiting for finishing the thread  
    try {  
        thread.join();  
    } catch (InterruptedException e) {  
        e.printStackTrace();  
    }  
}
```

OUTPUT:-



---

4. Write a Java program that creates and starts three threads. Each thread should print its name and count from 1 to 5 with a delay of 500 milliseconds between each count.

CODE:- **package** LAB\_8;

```
public class ThreadDemo {  
    public static void main(String[] args) {  
        // Create and start three threads  
        Thread thread1 = new Thread(new CountingTask("Thread-1"));  
        Thread thread2 = new Thread(new CountingTask("Thread-2"));  
        Thread thread3 = new Thread(new CountingTask("Thread-3"));  
  
        thread1.start();  
        thread2.start();  
        thread3.start();  
    }  
}
```

```

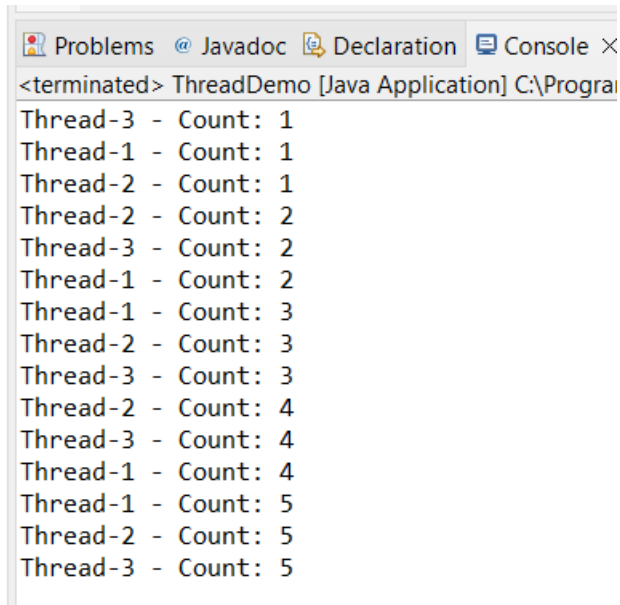
// Runnable implementation to count and print with a delay
class CountingTask implements Runnable {
    private String threadName;

    public CountingTask(String name) {
        this.threadName = name;
    }

    @Override
    public void run() {
        // Count from 1 to 5 with a delay of 500 milliseconds
        for (int i = 1; i <= 5; i++) {
            System.out.println(threadName + " - Count: " + i);
            try {
                // Delay of 500 milliseconds
                Thread.sleep(500);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

```

OUTPUT:-



```
<terminated> ThreadDemo [Java Application] C:\Program
Thread-3 - Count: 1
Thread-1 - Count: 1
Thread-2 - Count: 1
Thread-2 - Count: 2
Thread-3 - Count: 2
Thread-1 - Count: 2
Thread-1 - Count: 3
Thread-2 - Count: 3
Thread-3 - Count: 3
Thread-2 - Count: 4
Thread-3 - Count: 4
Thread-1 - Count: 4
Thread-1 - Count: 5
Thread-2 - Count: 5
Thread-3 - Count: 5
```

---

5. Create a Java program that demonstrates thread priorities. Create three threads with different priorities and observe the order in which they execute.

CODE:- **package** MultingThread;

**class** MyThread **extends** Thread

{

**public** MyThread(String name, **int** priority)

    {

**super**(name);

        setPriority(priority);

    }

**public void** run()

    {

**for** (**int** i=1;i<=5;i++)

        {

            System.**out**.println(getName() +": count "+ i);

```

        try
        {
            Thread.sleep(100);
        } catch (InterruptedException e)
        {
            System.err.println(e);
        }
    }
}

public class ThreadPriorityDemo {

    public static void main(String[] args) {

        MyThread thread1=new MyThread("high priority
thread",Thread.MAX_PRIORITY);

        MyThread thread2=new MyThread("medium priority
thread",Thread.NORM_PRIORITY);

        MyThread thread3=new MyThread("low priority
thread",Thread.MIN_PRIORITY);

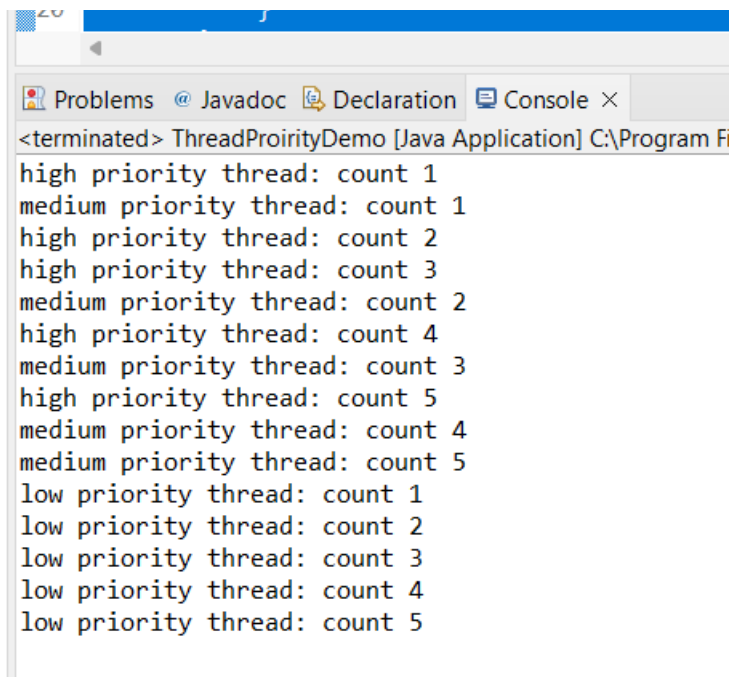
        thread1.start();
        thread2.start();
        thread3.start();

    }

}

```

OUTPUT:-



```
<terminated> ThreadPriorityDemo [Java Application] C:\Program Fi
high priority thread: count 1
medium priority thread: count 1
high priority thread: count 2
high priority thread: count 3
medium priority thread: count 2
high priority thread: count 4
medium priority thread: count 3
high priority thread: count 5
medium priority thread: count 4
medium priority thread: count 5
low priority thread: count 1
low priority thread: count 2
low priority thread: count 3
low priority thread: count 4
low priority thread: count 5
```